

엠버 폴

김승범, 김성준, 정영기
한국공학대학교 게임공학과

wewe231223@tukorea.ac.kr, lastbabo2480@tukorea.ac.kr, rldwdwjd123@tukorea.ac.kr

요 약

본 논문은 현대 그래픽스 API인 DirectX12와 최신 렌더링 기술인 Mesh Shader 및 Volumetric Fog를 활용하여 개발된 협동 액션 RPG (Cooperative Action RPG)의 개발 과정과 핵심 기술을 서술한다. 또한, 고성능 멀티쓰레딩 서버 구현을 위해 IOCP(I/O Completion Port) 모델을 적용하여 안정적인 다중 사용자 환경을 구축하는 방법을 제시한다. 본 게임은 비대칭적인 역할 분담(Player vs. Boss)을 통해 기존 액션 RPG와 차별화된 깊이 있는 협동 및 경쟁 경험을 제공하는 것을 목표로 한다. 본 연구를 통해 고성능 게임 개발에 필요한 클라이언트 및 서버 기술의 통합적 이해와 실제 적용 방안을 제시한다.

1. 서 론

1.1 게임 제작 목적 및 기획 의도

최근 게임 시장은 사용자 간의 협력 (Cooperation)과 경쟁(Competition)이 혼합된 형태의 인터랙티브 콘텐츠에 대한 수요가 증가하고 있다. 본 게임은 이러한 시장의 요구를 반영하여, 단순한 PVE(Player vs. Environment)나 PVP(Player vs. Player)를 넘어선 Player와 Boss 역할군을 분리하여 플레이할 수 있는 비대칭적인 협동 액션 RPG를 개발하는 것을 목적으로 한다.

기획 의도는 플레이어들에게 'Boss' 역할의 강력한 힘과 'Player' 역할의 기민한 협동을 동시에 경험하게 함으로써, 하나의 게임 내에서 두 가지 상반된 전투 경험을 제공하는 데 있다. 특히, Boss 플레이어는 다수의 일반 플레이어를 상대로 전략적 우위를 점하고, 일반 플레이어들은 제한된 자원과 역할 분담을 통해 목표 달성이라는 공동의 미션을 수행하며

진정한 협동의 가치를 느끼도록 설계되었다.

항목	내용
플랫폼	PC
장르	협동 액션 RPG (Co-op Action RPG), 비대칭 멀티플레이
시점	3인칭 (TPS, Third-Person Shooter)
핵심 기술	DirectX12, Mesh Shader, Volumetric Fog, IOCP 기반 멀티쓰레드 서버
주요 특징	역할군 기반 협동 플레이, 대형 환경의 고성능 렌더링

[표 1] 게임 주요 특성 및 일반 정보

2. 개발 내용

2.1 게임 전체 흐름 및 콘텐츠

본 게임의 핵심 콘텐츠는 'Player' 역할과 'Boss' 역할 간의 비대칭적인 전투 경험에 중점을 둔다.

게임 플레이 흐름:

① 매칭 및 역할 선택: 플레이어들은 일반 Player 또는 강력한 Boss 중 하나의 역할을 선택한다.

② 1 스테이지 시작:

일반 플레이어들은 주어진 대형 맵 내에서 흰색 안개를 방출하는 역병 보석을 모두 파괴해야 한다.

Boss 플레이어는 일반 플레이어들의 목표 달성을 저지하고 모두 제거해야 한다.

③ 2 스테이지 시작:

일반 플레이어들은 Boss 플레이어를 찾아 협동하여 Boss 플레이어에게 맞서거나, Boss 플레이어의 시선을 끌며 팀 동료가 Boss 플레이어를 제거할 수 있도록 협동해야 한다.

Boss 플레이어는 이전 스테이지와 같이 일반 플레이어들의 목표 달성을 저지하고 모두 제거해야 한다.

승리 조건:

Player 팀: 모든 역병 보석을 파괴하고, Boss 플레이어를 제거.

Boss 팀: 제한 시간 내에 모든 Player를 제거.

2.2 개발 방법론: 기술 아키텍처

본 게임은 클라이언트 측 성능 최적화와 서버 측 안정성 및 확장성 확보를 위해 다음과 같은 기술 아키텍처를 채택하였다.

2.2.1 DirectX12 기반 클라이언트 개발

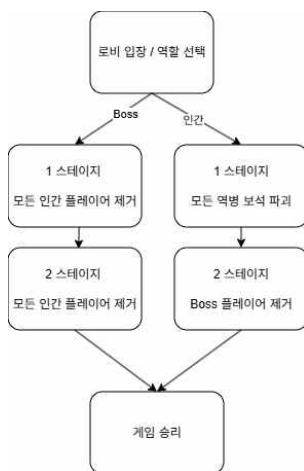
DirectX12는 이전 버전 대비 낮은 수준의 하드웨어 접근성(Low-Level API)을 제공하여, 드라이버 오버헤드를 줄이고 CPU 병렬 처리를 극대화한다. 이를 통해 멀티코어 환경에서 효율적인 Command List 생성 및 제출이 가능하며, 이는 특히 대규모 환경 렌더링에서 성능 향상에 결정적인 역할을 한다.

본 게임에서는 이러한 Command List의 병렬 처리 기능을 게임 리소스 로딩에 사용하여, 게임 최초 실행 시간을 단축하고, 병렬 처리를 통해 사용자 대기 시간을 단축하였다.

2.2.2 Mesh Shader를 활용한 대형 환경 인스턴스

대형 환경 식생을 효율적으로 렌더링하기 위해 기존의 Vertex Shader 및 Geometry Shader 파이프라인 대신 Mesh Shader 파이프라인을 도입하였다.

기존 파이프라인은 Draw Call당 처리할 수 있는 정점(Vertex)의 수가 제한적이며, 복잡한 지오메트리 데이터를 GPU에 비효율적으로 전달하는 한계가 있었다.



[그림 1] 게임 진행 흐름도

Mesh Shader는 GPU 기반 컬링(Culling) 및 LOD(Level of Detail) 관리를 가능하게 한다. 이 구조는 특히 나무, 바위 등 반복되는 대형 환경 오브젝트를 수천 개 인스턴싱할 때 CPU 부하를 획기적으로 줄이고 GPU의 효율을 극대화한다.

잔디 오브젝트의 구조를 XZ 평면 상의 위치를 기록한 Buffer를 사용하여, 3차원 위치를 사용하였을 때 보다 Buffer 구조를 축소하였다. 잔디를 실외 지형 위에 올리는 작업은 Mesh Shader에서 진행한다.

카메라와의 거리에 비례하여 선형적으로 잔디 오브젝트의 크기가 작아지면서 최종 거리부터는 Cull 되도록 만들어 오브젝트가 갑자기 사라지거나 나타나는 현상을 방지하였다.

이를 통해 잔디를 CPU에서 컬링하고, 이를 Geometry Shader 파이프라인을 사용하는 것 보다 성능을 약 2배 상승하는 효과를 확인하였다.



[그림 2] Mesh Shader를 활용한 잔디 인스턴스

2.2.3 Volumetric Fog를 사용한 현실감 있는 안개 구현

게임 환경의 몰입도 및 분위기 조성을 위하여 체적 안개(Volumetric Fog) 기술을 구현하였다[1]. 본 기술은 기존의 단순 거리 비례

안개(Exponential Fog) 기법의 한계를 극복하고, 광원 상호작용 및 공간적 깊이감을 현실적으로 표현하는 데 중점을 두었다.

기존 지수형 안개(Exponential Fog)는 카메라와의 거리에 비례하여 단순 색상 혼합을 수행하는 방식으로, 깊이감이나 빛의 분산 및 흡수에 따른 광원 상호작용 측면에서 현실적인 표현이 미흡하였다. 또한, 거리 비례 방식의 안개 구현은 안개의 탁색(濁色)만을 표현할 뿐, 공간을 채우는 볼륨감(Volumetric Quality)을 제공하지 못하며, 공기보다 무거워 하강하는 안개의 물리적 특성 또한 반영되지 못하는 근본적인 한계가 존재하였다.

체적 안개 구현을 위하여 3D 텍스처(Volume Texture)를 시야 절두체(View Frustum)에 매핑하는 방식을 채택하였다. 이는 3D 텍스처의 각 복셀(Voxel) 영역이 시야 절두체의 분할된 영역에 대한 안개의 밀도(Density) 및 색상 정보를 저장하고, 이를 누적하도록 설계됨을 의미한다.

최종적으로 이 누적된 텍스처에 저장된 색상 값이 카메라에 표현될 최종 안개의 색상이 되며, 해당 텍스처와 백 버퍼(Back Buffer)를 블렌딩(Blending) 처리함으로써 높은 현실감을 갖춘 체적 안개 효과를 성공적으로 표현하였다.

2.2.4 IOCP를 활용한 멀티쓰레드 서버 구현

안정적이고 고성능의 다중 사용자 환경을 지원하기 위해 IOCP(I/O Completion Port) 기반의 서버 아키텍처를 채택하였다.

Windows 환경에서 비동기 I/O 요청을 효율적으로 처리하는 메커니즘이다. 쓰레드를 미리 생성하고 I/O 요청이 완료될 때만 이



[그림 3] Volumetric Fog 적용 환경

쓰레드들을 사용하여, 유힘(Idle) 상태의 쓰레드 낭비를 최소화하고 쓰레드 컨텍스트 스위칭 오버헤드를 줄인다.

Worker Thread Pool을 생성하고, 각 Worker Thread는 GetQueuedCompletionStatus() 함수를 통해 I/O 완료 이벤트를 기다린다.

네트워크 I/O(Accept, Recv, Send)는 비동기 함수(예: WSARcv, WSASend)를 사용하여 IOCP에 바인딩한다.

I/O 작업이 완료되면 시스템이 Completion Queue에 결과를 등록하고, Worker Thread가 이를 처리한다.

이러한 구조는 수백 명의 동시 접속자를 안정적으로 처리할 수 있는 확장성(Scalability) 높은 서버 환경을 제공한다.

2.3 게임 진행 방법 및 협동 요소

역할군 분담 및 진행 방법:

Player 역할: 각자 다른 무기를 선택하여 주로 협동을 통해 목표를 완수하거나 Boss의 약점을 공략한다. (예: 한 명은 Boss의 시선을 끌고, 다른 한 명은 목표물을 활성화)

Boss 역할: 압도적인 체력, 공격력을 보유한다. Boss는 맵 전체를 감시하고, Player들의 움직임을 예측하여 미션 달성을 방해하는

데 중점을 둔다.

기대 효과: Player와 Boss 역할을 모두 경험함으로써 사용자들은 한 게임 내에서 두 가지 극단적인 전투 경험과 전략적 사고를 즐길 수 있다. 이는 게임의 재플레이 가치와 시장성을 높이는 핵심 요소이다.

3. 결 론

3.1 개발된 작품 요약

본 졸업 작품은 DirectX12 API를 기반으로 Mesh Shader를 활용한 고효율 대형 환경 렌더링과 Volumetric Fog를 통한 사실적인 시각적 몰입감을 제공하는 클라이언트와, IOCP를 활용하여 수백 명의 동시 접속을 안정적으로 처리할 수 있는 멀티쓰레드 서버를 통합한 협동 액션 RPG이다. 게임은 Player와 Boss 간의 비대칭적 전투 구조를 통해 독창적인 협동 및 경쟁의 재미를 구현하였다.

3.2 장단점 분석 및 평가

본 작품의 주요 강점은 기술적 완성도와 독창적인 콘텐츠 구조에서 찾을 수 있다. 기술적으로는 Mesh Shader를 활용하여 대규모 환경을 효율적으로 렌더링함으로써 최적화된 성능을 확보했으며, 서버 측면에서는 IOCP 기반의 멀티쓰레드 아키텍처를 채택하여 고성능과 높은 확장성을 동시에 실현했다. 콘텐츠 측면에서는 Player 대 Boss의 비대칭적 역할 분담 구조가 독창성을 제공하며, Player와 Boss 역할을 모두 경험할 수 있어 게임의 재플레이 가치와 다양한 전투 경험을 제공한다. 이는 강점이 있다.

반면, 본 작품은 몇 가지 한계점도 내포한다. 최신 기술인 Mesh Shader 및 DirectX12를 사용함에 따라 게임의 하드웨어 요구 사양

이 비교적 높게 설정되어 사용자 접근성이 제한될 수 있으며, 이는 시장성 측면에서도 제약 요인으로 작용한다. 또한, 고도화된 기술을 구현하는 과정에서 개발 난이도가 높았으며, 초기 개발 단계에서 콘텐츠(맵, 미션, 캐릭터 스킬)의 양이 상대적으로 제한적일 수 있다는 점이 단점으로 꼽힌다.



[그림 3] 인간 플레이어 시점



[그림 4] Boss 플레이어 시점

4. 부 록

1) 매뉴얼 작성

(개발된 작품이 게임이 아닌 경우는 그 프로그램을 설치 및 실행하는데 필요한 모든 정보를 게임과 유사한 목차로 작성하기 바람)

4.1 게임 실행 환경:

본 작품은 최소 Windows 10 버전 이상이 요구된다. DirectX12와 DirectX 12 Tier 2

Feature Level을 만족하는 그래픽카드가 요구된다.

4.2 게임 설치:

클라이언트의 경우, 게임 파일 압축을 해제한 뒤, IPADDR.txt 파일에 서버의 주소를 기입한 뒤, EmberFall.exe 파일을 실행한다.

이름	수정된 날짜	유형	크기
Log	2025-07-28 오전 11:59	파일 폴더	
Resources	2025-07-22 오후 3:47	파일 폴더	
Shader	2025-07-22 오후 3:48	파일 폴더	
assimp-vc143-mt.dll	2025-05-01 오후 7:26	응용 프로그램 확장	5,447KB
D3D12Core.dll	2025-02-14 오전 3:39	응용 프로그램 확장	3,273KB
dxcompiler.dll	2025-05-01 오후 7:26	응용 프로그램 확장	17,566KB
EditorInterface.lib	2025-05-11 오후 9:27	Object File Library	19,770KB
EditorInterface.pdb	2025-05-11 오후 9:27	Program Debug ...	5,476KB
EmberFall.exe	2025-07-28 오전 11:49	응용 프로그램	1,318KB
EmberFall.pdb	2025-07-28 오전 11:49	Program Debug ...	24,620KB
fmod.dll	2025-07-22 오후 3:47	응용 프로그램 확장	1,833KB
Game.lib	2025-07-28 오전 11:48	Object File Library	101,867KB
Game.pdb	2025-07-28 오전 11:48	Program Debug ...	10,980KB
imgui.ini	2025-07-26 오후 4:53	구성 설정	1KB
IPADDR.txt	2025-07-28 오전 11:51	텍스트 문서	1KB
MeshLoader.lib	2025-07-28 오전 11:48	Object File Library	29,393KB
MeshLoader.pdb	2025-07-28 오전 11:48	Program Debug ...	2,700KB
Renderer.lib	2025-07-28 오전 11:48	Object File Library	93,183KB
Renderer.pdb	2025-07-28 오전 11:48	Program Debug ...	9,196KB
ServerLib.lib	2025-07-28 오전 11:48	Object File Library	8,788KB
ServerLib.pdb	2025-07-28 오전 11:48	Program Debug ...	3,652KB

[그림 5] 클라이언트 파일 내용

서버의 경우, 서버 파일 압축을 해제한 뒤, Executable 폴더 내에 있는 EmberFallServer.exe를 실행한다.

이름	수정된 날짜	유형	크기
Resources	2025-05-10 오후 2:18	파일 폴더	
Executable	2025-05-17 오후 5:10	파일 폴더	

[그림 6] 서버 파일 내용

4.3 게임 진행

4.3.1 게임 전체 흐름도

4.3.2 게임 진행 방법(인터페이스 설명, 캐릭터, 아이템, 미션, 게임 진행 예 등을 화면 캡처를 통해서 게임 진행 설명)²⁾

4.4 문제 해결

4.4.1 개발된 게임의 제약사항

4.4.2 문제 해결을 위한 연락처

5. 참고 문헌

[1] B. Wronski, "Volumetric fog: unified, compute shader based solution to atmospheric scattering," in ACM SIGGRAPH 2014 Talks, 2014, Art. no. 60. doi: 10.1145/2614068.2614126.